**LIMKOKWING UNIVERSITY OF CREATIVE TECHNOLOGY**

Sierra Leone

Department of Information and Communication Technology

**PROG211 – OBJECT-ORIENTED PROGRAMMING 1**

Individual Assignment (10%) – Real-World Solutions under DPG Standards

---

# CITIZEN COMPLAINT SYSTEM

---

*Domain: Government | System: Citizen Complaint System*

**Student Name:** [YOUR FULL NAME]

**Student ID:** [YOUR STUDENT ID]

**Programme / Class:** [YOUR PROGRAMME AND CLASS]

**Examiner:** Amandus Benjamin Coker

**Semester:** Semester 03, September 2026 to February 2027

**Submission Deadline:** Week 4

**GitHub Repository:** [PASTE YOUR GITHUB REPOSITORY LINK]

## Table of Contents

1. Introduction and Problem Statement
2. Chosen Domain and System
3. System Design
4. Task 1 – Classes, Attributes and Objects (Lecture 1)
5. Task 2 – Python Data Structure (Lecture 2)
6. Task 3 – Methods (Lecture 3)
7. Additional Features
8. Digital Public Goods (DPG) Standards Alignment
9. Program Execution (Screenshots)
10. Testing
11. GitHub Submission
12. Assignment Requirements Checklist
13. Conclusion and Future Improvements
14. References

Appendix A – Full Source Code

# 1. Introduction and Problem Statement

Good public services depend on governments hearing about problems quickly: a dangerous pothole, a broken water pipe, uncollected rubbish, a power fault or a health clinic that is not open when it should be. In many communities, however, citizens do not know where to report these problems. Complaints made by word of mouth are easily forgotten, and there is no record of how long problems take to fix. Citizens also worry that complaining could expose them to unwanted attention.

This assignment solves that real-world problem with a Python program called the **Citizen Complaint System**. Every complaint is routed automatically to the responsible department and given a reference number and a target resolution time. It then moves through a fixed lifecycle (Submitted → In progress → Resolved, or Rejected), and every change is recorded in a history that cannot be edited. Citizens can track and rate their complaints and may remain **completely anonymous**. Officials can see overdue complaints and compare how well departments perform. The program is built with the Object-Oriented Programming (OOP) concepts from Lectures 1 to 3 and follows Digital Public Goods (DPG) standards.

## 1.1 Aim and objectives

The aim is to make public services more transparent and accountable through a simple, free and privacy-respecting complaint tool. The objectives are to:

- choose one domain (Government) and one system (Citizen Complaint System) from the brief;
- create at least two classes with attributes and methods, and show objects being created and interacting (Task 1);
- store multiple objects using only one data structure, with functions to add and display records (Task 2);
- implement instance methods and class methods (Task 3), plus static methods;
- follow the DPG principles: open-source, inclusive and accessible, privacy-respecting, and modular and reusable;
- document the work in a README, this hardcopy report with screenshots, and a public GitHub repository.

# 2. Chosen Domain and System

The chosen domain is **Government** and the chosen system is the **Citizen Complaint System**. It was chosen because complaint handling links citizens directly to public institutions, and because it raises important questions of privacy and trust that the DPG principles address.

## 2.1 What the system can do

- Register citizens with a first name and district, or completely anonymously.
- File a complaint in one of five categories; it is routed to the right department and given a reference number and a target time based on its priority.
- Update a complaint's status, allowing only valid moves, and record each change with a date and note.
- Let citizens track their complaint and rate it from 1 to 5 stars once it is resolved.
- Withdraw (delete) a complaint, or delete a citizen together with all their complaints, after confirmation.

- Search complaints; show the best-performing department, a department performance report and a list of overdue complaints.

## 3. System Design

The whole solution is written in **one Python file**, `complaint_system.py`, divided into eight clearly labelled sections:

| Section | Purpose |
|---|---|
| 1 – Fixed data | Tuples: DISTRICTS (all 16 districts of Sierra Leone), CATEGORIES, PRIORITIES, STATUS_FLOW, OPEN_STATUSES and RATING_RANGE, with target_days() and category_details(). |
| 2 – Classes | Citizen, Department and Complaint. |
| 3 – Record functions | add_record(), find_record(), withdraw_complaint(), delete_citizen(), display_complaints() and search_complaints(). |
| 4 – Sample data | load_sample_data() creates fictional citizens, complaints, status updates and ratings. |
| 5 – Input helpers | ask(), ask_until_valid() and small validators: every answer is checked as it is typed. |
| 6 – Menu actions | One function for each of the 13 menu options. |
| 7 – Menu and demo | main() is the interactive menu; run_demo() is a scripted walk-through used for the screenshots. |
| 8 – Unit tests | Ten unit tests that check the classes and rules. |

The program runs with `python complaint_system.py` (menu), `python complaint_system.py --demo` (demonstration) or `python complaint_system.py --test` (unit tests). Only the Python standard library is used.

### 3.1 Classes and their relationships

| Class | Attributes | Main methods |
|---|---|---|
| Citizen | citizen_id, first_name, district, complaints (dictionary)<br>Class attributes: total_citizens, next_number | add_complaint(), open_complaints(), display_info(), register() [class], register_anonymous() [class], get_total_citizens() [class], record_removal() [class], normalise_district() [static], is_valid_district() [static] |
| Department | code, name, complaints (dictionary) | receive(), count_by_status(), resolution_rate(), average_rating(), average_days_to_resolve(), on_time_rate(), performance_score(), statistics(), from_category() [class], find_best() [static] |

| Class | Attributes | Main methods |
|---|---|---|
| Complaint | reference, citizen, department, category, description, priority, date_filed, status, date_closed, rating, history (tuple)<br>Class attributes: total_complaints, next_number | is_open(), days_open(), is_overdue(), update_status(), rate(), display_row(), display_details(), from_text() [class], get_total_complaints() [class], record_removal() [class], normalise_priority() [static], is_valid_rating() [static], parse_date() [static] |

### 3.2 Complaint lifecycle

Figure 1 shows the lifecycle that every complaint follows. The allowed moves are stored in the `STATUS_FLOW` tuple, and `update_status()` refuses any move that is not in it. A complaint therefore cannot jump straight from Submitted to Resolved, and a closed complaint cannot be reopened.

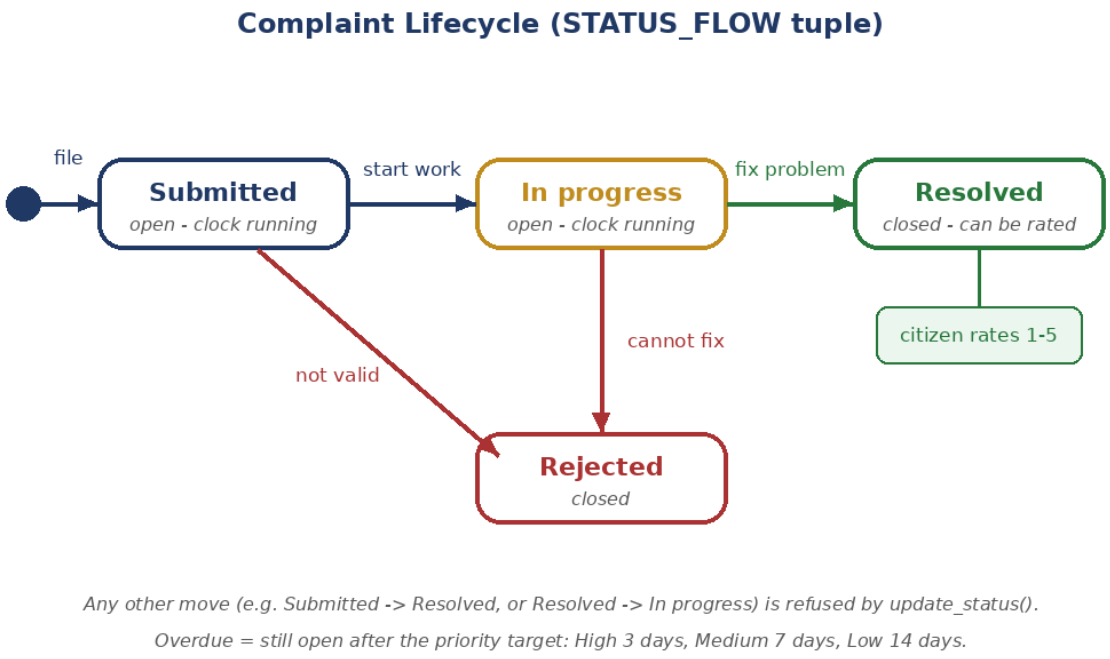



***Figure 1:*** *The complaint lifecycle enforced by update_status() and the STATUS_FLOW tuple*

## 4. Task 1 – Classes, Attributes and Objects (Lecture 1)

The brief asks for **at least two classes** relevant to the chosen domain, with **attributes** and **methods**, and a demonstration of **object creation and interaction**. This project uses **three** classes: Citizen, Department and Complaint. Each sets its instance attributes in `__init__`, has methods that act on those attributes, and uses class attributes for counting and numbering.

### 4.1 Example: the Citizen class

Only a first name and a district are kept. A citizen who chooses not to give a name is stored as "Anonymous":

```
class Citizen:
```

```
    """A person who files complaints. May be anonymous."""

    total_citizens = 0
    next_number = 1

    def __init__(self, citizen_id, first_name, district):
        if not Citizen.is_valid_district(district):
            raise ValueError("Please choose one of the 16 districts of Sierra Leone.")
        self.citizen_id = citizen_id.upper()
        self.first_name = first_name.strip().capitalize() or "Anonymous"
        self.district = Citizen.normalise_district(district)
        self.complaints = {}                # {reference: Complaint}
        Citizen.total_citizens += 1

    # ---------- instance methods ----------
```

## 4.2 Object creation

```
Citizen.register("Aminata", "Western Area Urban")       # automatic ID C001
Citizen.register_anonymous("Kenema")                      # no name stored
Department.from_category(("ROAD", "Roads and potholes", "DPW", "Public Works Department"))
Complaint.from_text("C001, ROAD, Large pothole near the market, High, 2026-09-01", citizens,
departments)
```

## 4.3 Interaction between objects

The Complaint constructor shows the three classes working together. A complaint keeps references to its Citizen and Department objects and registers itself with **both**. The citizen can then count its open complaints, and the department can calculate its statistics from the complaints it holds:

```
def __init__(self, citizen, department, category, description, priority, date_filed):
    if not description.strip():
        raise ValueError("Please describe the problem.")
    self.priority = Complaint.normalise_priority(priority)
    if isinstance(date_filed, str):
        date_filed = Complaint.parse_date(date_filed)
    self.reference = f"CMP-{Complaint.next_number:04d}"
    Complaint.next_number += 1
    Complaint.total_complaints += 1
    self.citizen = citizen               # Citizen OBJECT
    self.department = department         # Department OBJECT
    self.category = category.upper()
    self.description = description.strip()
    self.date_filed = date_filed
    self.status = "Submitted"
    self.date_closed = None
    self.rating = None
    # history is a tuple of (date, status, note) tuples
    self.history = ((date_filed, "Submitted", "Complaint received"),)
    # objects interacting: register with the citizen AND the department
    citizen.add_complaint(self)
    department.receive(self)
```

The Department then works out its own performance from its Complaint objects:

```
def on_time_rate(self):
    """Percentage of resolved complaints that were resolved within their target days."""
    on_time, resolved = 0, 0
    for complaint in self.complaints.values():
        if complaint.status == "Resolved":
            resolved += 1
            if complaint.days_open() <= target_days(complaint.priority):
                on_time += 1
    return round(on_time / resolved * 100) if resolved else 0

def performance_score(self):
    """Out of 100: 50% resolution rate + 30% citizen rating + 20% resolved on time."""
    rating_percent = self.average_rating() / RATING_RANGE[1] * 100
    return round(0.5 * self.resolution_rate() + 0.3 * rating_percent
                 + 0.2 * self.on_time_rate())
```

## 5. Task 2 – Python Data Structure (Lecture 2)

The brief requires **only one data structure, either a list or a dictionary,** to store multiple objects, with simple functions to **add** and **display** records. This project uses **dictionaries** throughout, with the unique ID as the key:

```
citizens    = {"C001": <Citizen>, "C002": <Citizen>, ...}
departments = {"DPW": <Department>, "DWS": <Department>, ...}
complaints  = {"CMP-0001": <Complaint>, "CMP-0002": <Complaint>, ...}
citizen.complaints = {"CMP-0001": <Complaint>, ...}
```

### 5.1 Why a dictionary was chosen

A citizen tracking a complaint only knows its reference number. A dictionary keyed by reference finds it instantly, without searching through every complaint, and duplicate references are impossible because keys are unique. To respect the one-data-structure rule, **every collection of objects is a dictionary**: no list or set is used to store project objects.

### 5.2 Functions to add, display and delete records

```
def add_record(records, key, item):
    """Add an object to a dictionary (duplicate keys are refused)."""
    if key in records:
        raise ValueError(f"{key} already exists.")
    records[key] = item
    return item


def display_complaints(complaints):
    print(f"\n  ALL COMPLAINTS ({len(complaints)})")
    print("  " + "-" * 72)
    print(f"  {'Reference':<10}{'Type':<8}{'District':<20}{'Priority':<9}{'Status':<13}
{'Open':>6}")
    print("  " + "-" * 72)
    for complaint in complaints.values():
        complaint.display_row()


def withdraw_complaint(complaints, reference):
    """Delete one complaint everywhere it is stored."""
    complaint = find_record(complaints, reference, "complaint")
    del complaints[complaint.reference]
    del complaint.citizen.complaints[complaint.reference]
    del complaint.department.complaints[complaint.reference]
    Complaint.record_removal()
    return complaint


def delete_citizen(citizens, complaints, citizen_id):
    """Delete a citizen and every complaint they filed."""
    citizen = find_record(citizens, citizen_id, "citizen")
    for reference in tuple(citizen.complaints):        # copy keys before deleting
        withdraw_complaint(complaints, reference)
    del citizens[citizen.citizen_id]
    Citizen.record_removal()
    return citizen
```

### 5.3 Optional bonus: correct use of tuples

Tuples are used **only for data that must not change** while the program runs:

```
# The 16 districts of Sierra Leone
DISTRICTS = (
    "Bo", "Bombali", "Bonthe", "Falaba", "Kailahun", "Kambia", "Karene", "Kenema",
    "Koinadugu", "Kono", "Moyamba", "Port Loko", "Pujehun", "Tonkolili",
    "Western Area Rural", "Western Area Urban",
)

# (category code, description, responsible department code, department name)
CATEGORIES = (
    ("ROAD",   "Roads and potholes",     "DPW", "Public Works Department"),
    ("WATER",  "Water supply",           "DWS", "Water Services Department"),
    ("POWER",  "Electricity supply",     "DEL", "Electricity Department"),
```

```
    ("WASTE",  "Waste collection",       "DSN", "Sanitation Department"),
    ("HEALTH", "Health facility service", "DHS", "Health Services Department"),
)

# (priority, target number of days to resolve)
PRIORITIES = (
    ("High",   3),
    ("Medium", 7),
    ("Low",   14),
)

# Allowed status changes: (from status, to status)
STATUS_FLOW = (
    ("Submitted",   "In progress"),
    ("Submitted",   "Rejected"),
    ("In progress", "Resolved"),
    ("In progress", "Rejected"),
)
OPEN_STATUSES = ("Submitted", "In progress")

RATING_RANGE = (1, 5)   # citizen satisfaction stars
```

Tuples are also used in several other places:

- **Complaint history:** each complaint's `history` is a tuple of `(date, status, note)` tuples. Each update builds a new tuple, so earlier entries can never be edited. This gives a trustworthy audit trail, which matters in government.
- **Returned values:** `Department.statistics()` returns an eight-value tuple, and `category_details()` returns a four-value tuple.
- **The menu labels.**
- **Tuple unpacking:** for example `for name, days in PRIORITIES`, `for when, status, note in self.history` and `total, open_count, resolved, rate, rating, days, on_time, score = department.statistics()`.

## 6. Task 3 – Methods (Lecture 3)

The brief requires **at least one instance method** and **at least one class method** (`@classmethod`). The overview also mentions **static methods**, so all three kinds are used:

| Type | Method | What it does |
|---|---|---|
| Instance | Complaint.update_status() | Moves one complaint along the lifecycle, checking STATUS_FLOW, and adds to its history. |
| Instance | Complaint.rate() | Records a 1–5 rating, allowed only when resolved. |
| Instance | Complaint.is_overdue(), days_open() | Check one complaint against its priority target. |
| Instance | Department.performance_score(), statistics() | One department's score and statistics. |
| Instance | Citizen.open_complaints() | Counts one citizen's open complaints. |
| Class | Citizen.register() | Creates a citizen with an automatic ID. |
| Class | Citizen.register_anonymous() | Creates an anonymous citizen (calls cls.register). |

| Type | Method | What it does |
|---|---|---|
| Class | Complaint.from_text() | Builds a complaint from one line of text and routes it to the right department. |
| Class | Department.from_category() | Builds a department from a CATEGORIES tuple. |
| Class | get_total_citizens(), get_total_complaints(), record_removal() | Read and update the shared counters. |
| Static | Citizen.is_valid_district(), normalise_district() | Check a district in any capitalisation. |
| Static | Complaint.normalise_priority(), is_valid_rating(), parse_date() | Check priorities, ratings and dates (no future dates). |
| Static | Department.find_best() | Finds the best-performing department. |

## 6.1 Instance methods

```
def update_status(self, new_status, note, when=None):
    """Move the complaint to a new status, following STATUS_FLOW."""
    new_status = new_status.strip().capitalize()
    if (self.status, new_status) not in STATUS_FLOW:
        raise ValueError(f"A complaint that is '{self.status}' cannot become '{new_status}'.")
    when = when or date.today()
    if isinstance(when, str):
        when = Complaint.parse_date(when)
    if when < self.date_filed:
        raise ValueError("The update date cannot be before the complaint was filed.")
    old = self.status
    self.status = new_status
    if not self.is_open():
        self.date_closed = when
    self.history = self.history + ((when, new_status, note.strip() or "-"),)
    print(f"  {self.reference}: {old} -> {new_status}")

def rate(self, stars):
    """Citizen satisfaction rating, only allowed once the complaint is resolved."""
    if self.status != "Resolved":
        raise ValueError("Only resolved complaints can be rated.")
    if not Complaint.is_valid_rating(stars):
        low, high = RATING_RANGE
        raise ValueError(f"Rating must be a whole number from {low} to {high}.")
    self.rating = stars
```

## 6.2 Class methods

```
@classmethod
def register(cls, first_name, district):
    """Create a citizen with an automatic ID such as C004."""
    citizen = cls(f"C{cls.next_number:03d}", first_name, district)
    cls.next_number += 1
    return citizen

@classmethod
def register_anonymous(cls, district):
    """Create an anonymous citizen - only the district is kept."""
    return cls.register("Anonymous", district)

@classmethod
def from_text(cls, text, citizens, departments):
    """Create a complaint from 'C001, ROAD, Large pothole, High, 2026-09-01'."""
    citizen_id, category, description, priority, when = (p.strip() for p in text.split(","))
    citizen = find_record(citizens, citizen_id, "citizen")
```

```
    _, _, dept_code, _ = category_details(category)
    return cls(citizen, departments[dept_code], category, description, priority, when)
```

### 6.3 Static methods

```
@staticmethod
def is_valid_district(text):
    return Citizen.normalise_district(text) is not None

@staticmethod
def find_best(departments):
    """Return the department with the highest performance score (or None)."""
    best = None
    for department in departments.values():
        if not department.complaints:
            continue
        if best is None or department.performance_score() > best.performance_score():
            best = department
    return best
```

## 7. Additional Features

| Feature (menu option) | Description |
|---|---|
| Withdraw a complaint (7) | Deletes one complaint from the register, the citizen and the department, after the user types YES. |
| Delete a citizen (8) | Deletes a citizen and every complaint they filed, after the user types YES. Counters are updated with class methods, and IDs and references are never reused. |
| Best-performing department (10) | The "top scorer". Score = 50% resolution rate + 30% citizen rating + 20% resolved within target, so speed counts as well as results. |
| Department performance report (11) | Total, open, resolved, resolution %, average rating, average days, % on time and score for every department. |
| Overdue complaints (12) | Every open complaint that has passed its priority target, and how many days late it is. |
| Search (9) | By reference, category, district, status or any word in the description. |
| Anonymous reporting (6) | Citizens can report without giving any name at all. |
| Lifecycle rules (3) | Invalid status changes are refused, and every change is dated in an unchangeable history. |

## 8. Digital Public Goods (DPG) Standards Alignment

| DPG principle | How this project meets it |
|---|---|
| Open-source | The full source code is published on GitHub under the MIT licence and uses only the Python standard library, so councils, ministries and civil-society groups can use, study, change and share it at no cost. |

| DPG principle | How this project meets it |
|---|---|
| Inclusive and accessible design | Works offline in any text terminal on a low-cost computer, for example at a council office or community centre. The menu uses plain English, and every prompt shows an example or a list of choices. Districts, categories, priorities and references are accepted in any capitalisation. Every answer is checked straight away and re-asked with a message starting 'Problem:', so the program never crashes. Status, overdue flags and ratings are shown in words and * symbols rather than colours. Deleting always needs a typed YES. |
| Privacy-respecting | Citizens can be completely anonymous; otherwise only a first name and district are stored. No surname, phone number, email, home address or national ID number is collected. Data stays on the local machine. Deleting a citizen removes all their complaints. All sample data is fictional. |
| Modular and reusable | Eight clearly separated sections, with one function per feature. A new complaint category and department is added with one line in CATEGORIES, and priority targets or allowed status changes are edited in their tuples, without changing the classes. The classes could be reused behind a web form, SMS service or mobile app. |

## 9. Program Execution (Screenshots)

Figures 2 to 4 show the scripted demonstration (`python complaint_system.py --demo`). Figures 5 to 11 show the interactive menu (`python complaint_system.py`), including how wrong input is handled. Figure 12 shows all the unit tests passing.

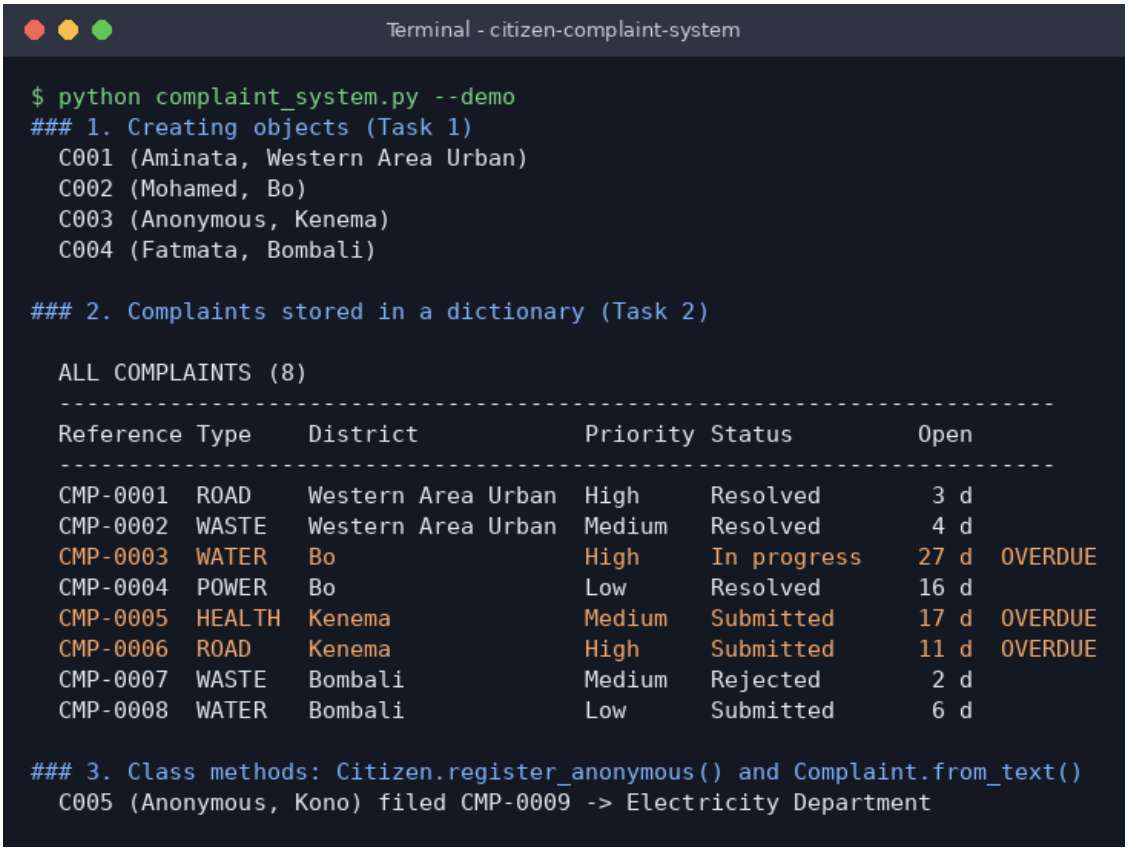



***Figure 2:*** *Demonstration (part 1): creating citizens, the complaints dictionary, and class methods*

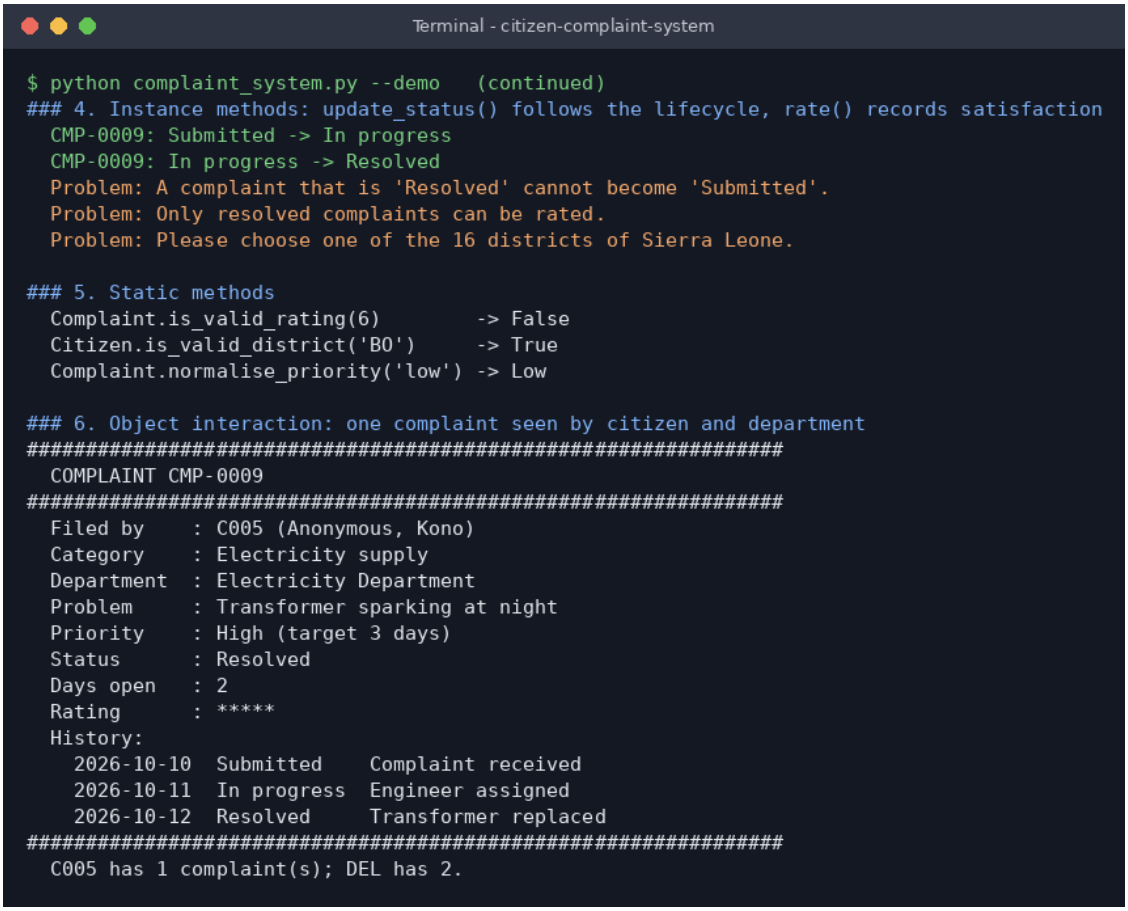



***Figure 3:*** *Demonstration (part 2): instance methods, lifecycle rules, static methods and a complaint seen by both its citizen and department*

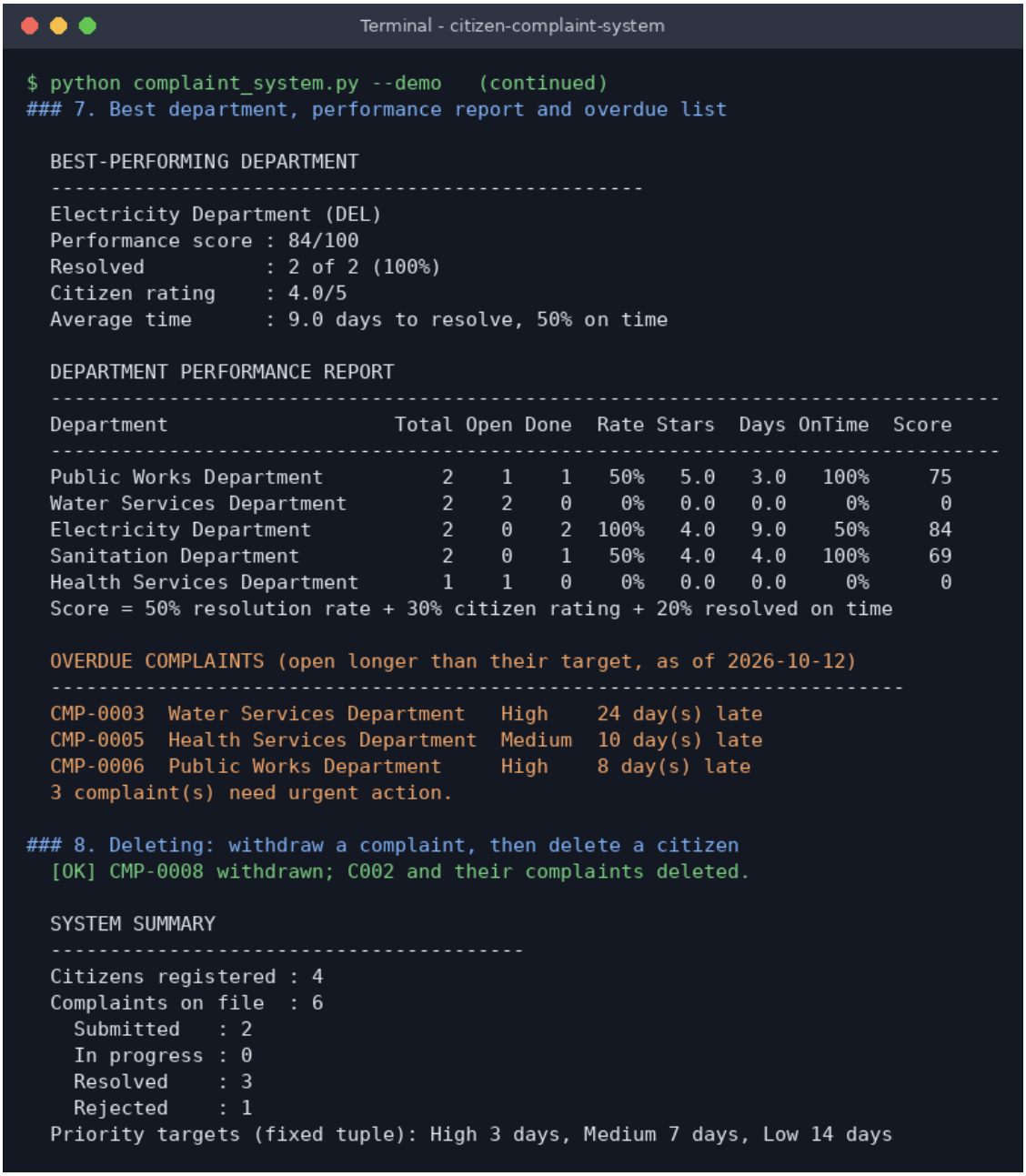



***Figure 4:*** *Demonstration (part 3): best department, performance report, overdue list, deleting and the summary*

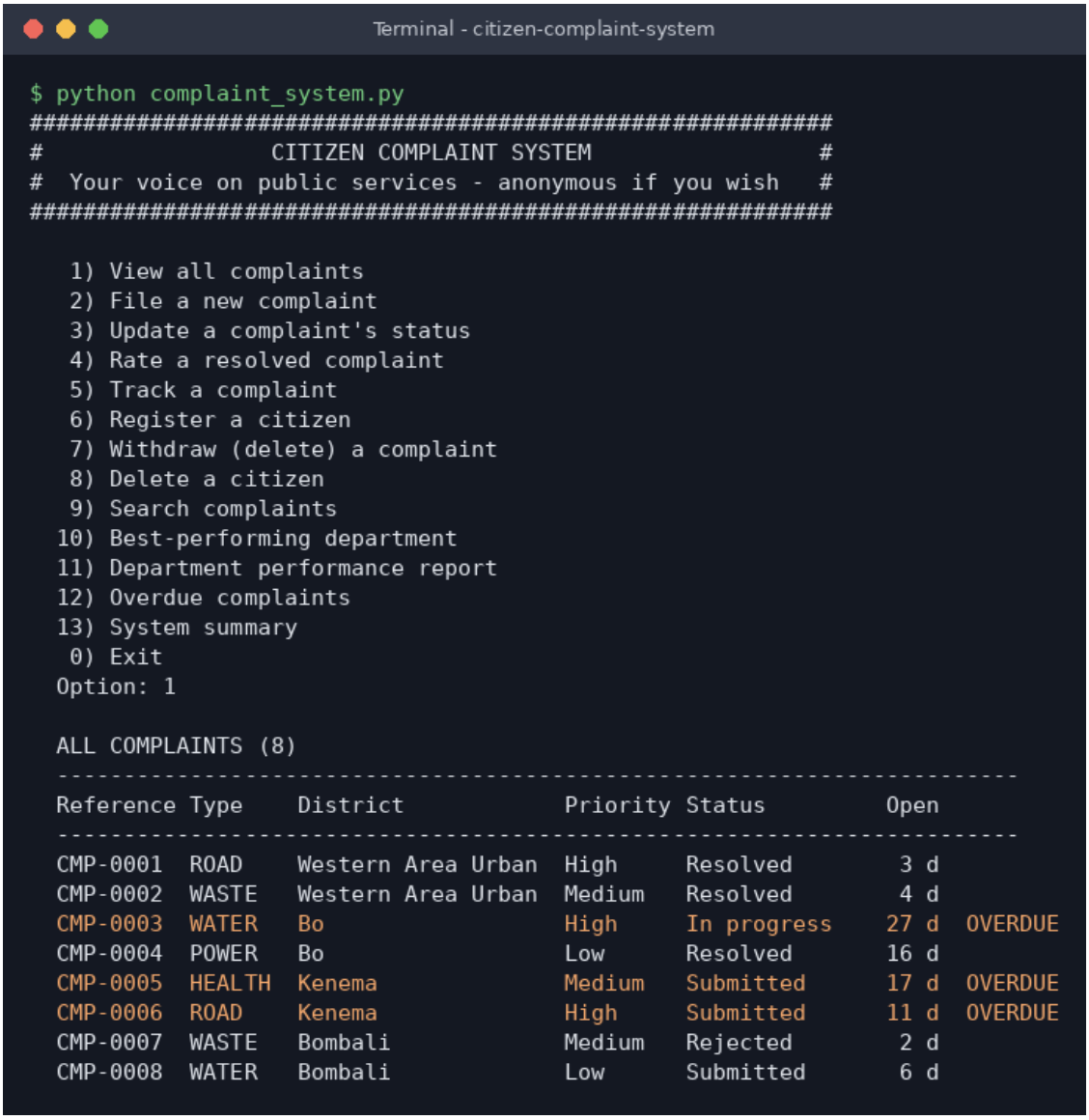



***Figure 5:*** *Option 1: all complaints displayed from the dictionary, with overdue flags*




***Figure 6:*** *Option 2: filing a complaint; an invalid category, empty description and invalid priority are re-asked*




***Figure 7:*** *Options 3 and 4: valid and invalid status changes, and rating a resolved complaint*

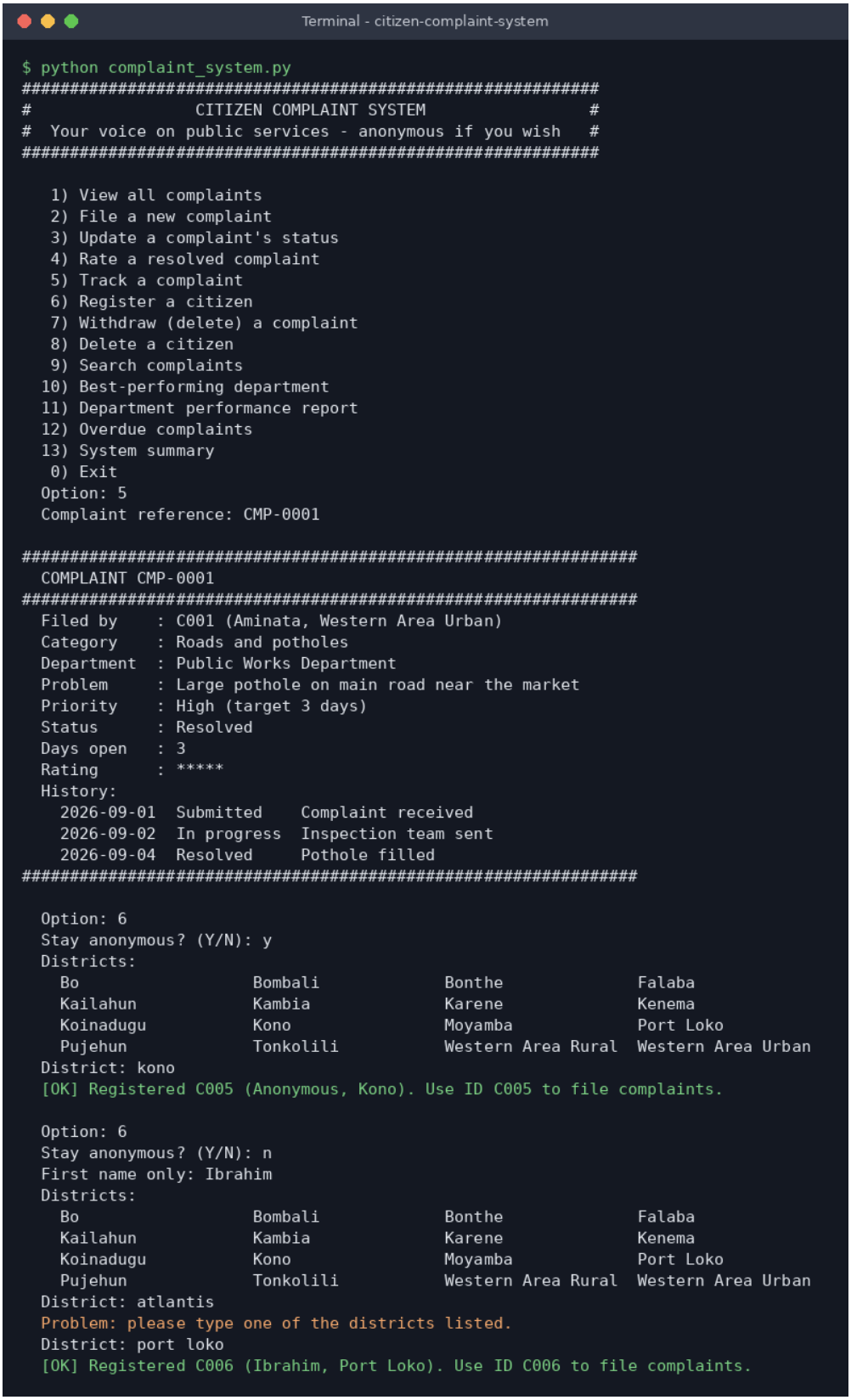



***Figure 8:*** *Options 5 and 6: tracking a complaint's history, and registering anonymous and named citizens*




***Figure 9:*** *Options 7, 8 and 9: a withdrawal cancelled then confirmed, a citizen deleted, and a search*

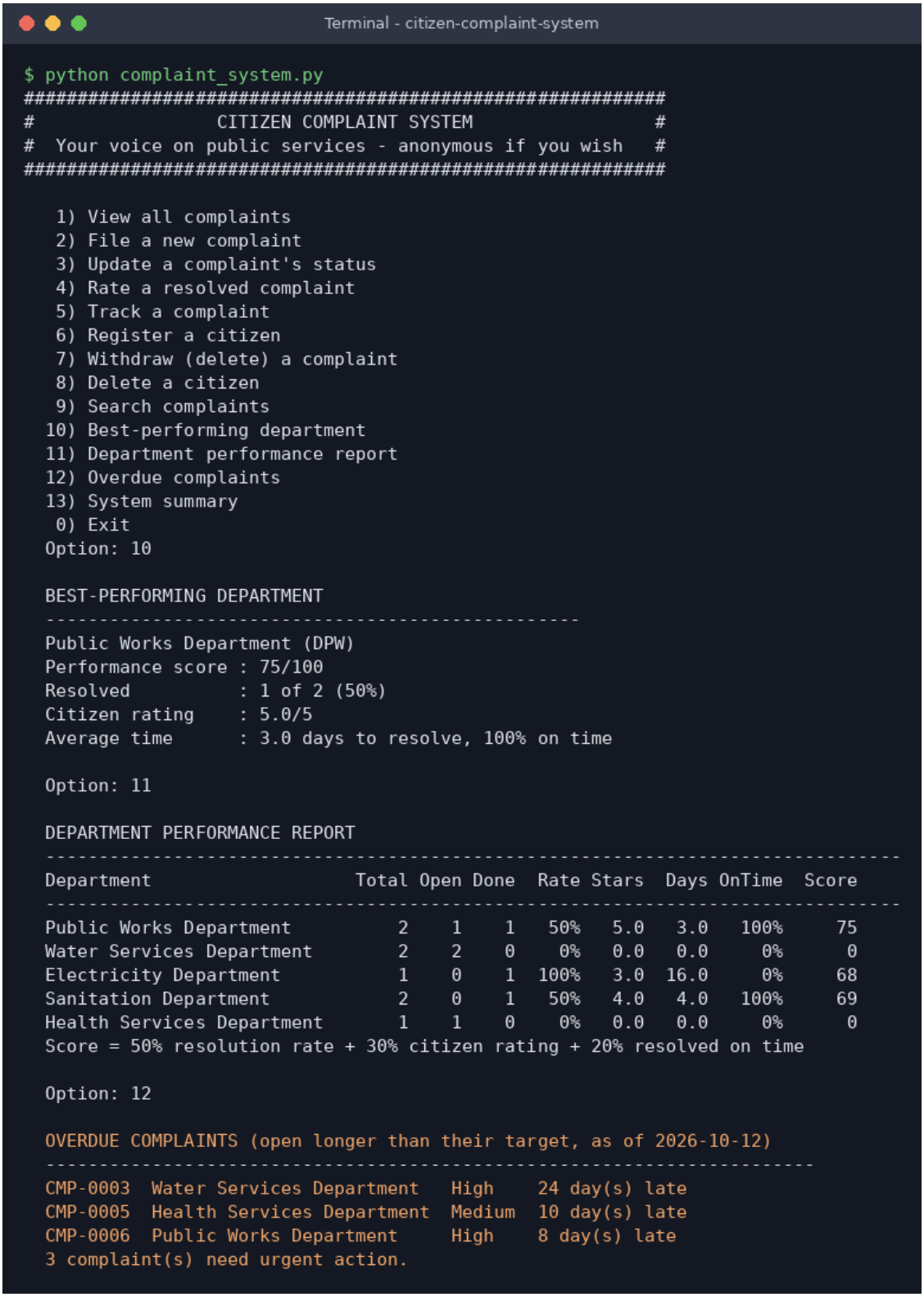



***Figure 10:*** *Options 10, 11 and 12: best department, performance report and overdue complaints*




***Figure 11:*** *Option 13: system summary, an invalid choice, and exit*

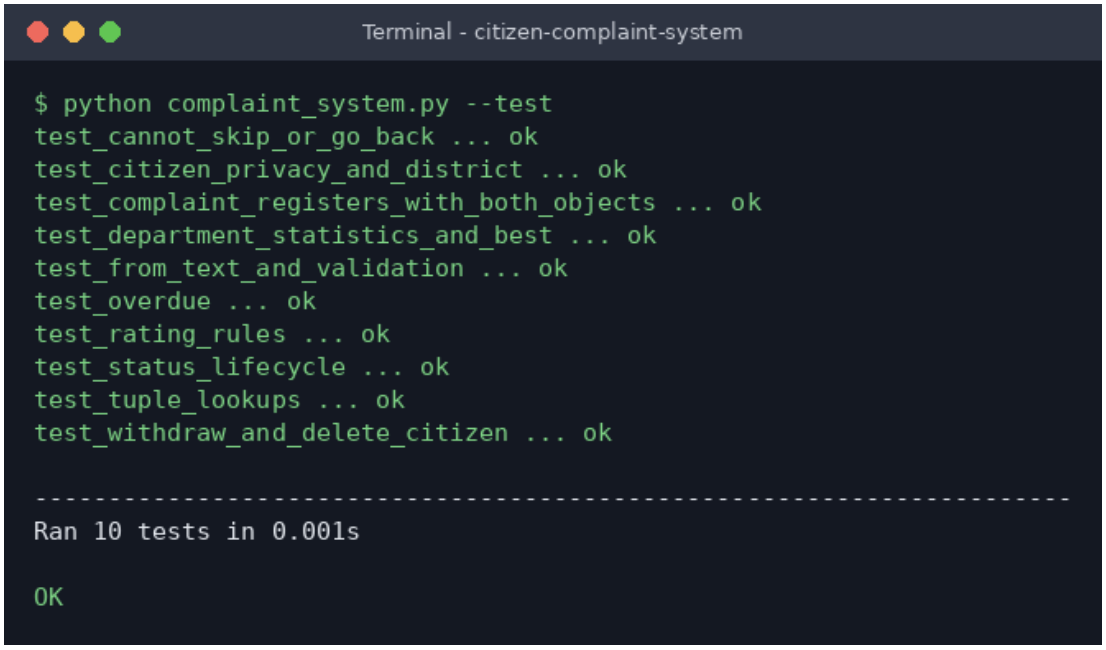



***Figure 12:*** *All 10 unit tests passing (python complaint_system.py --test)*

## 10. Testing

Ten unit tests were written with Python's built-in `unittest` module (Section 8 of the program). They check that:

- names are stored as first names only, anonymous registration works, and invalid districts are refused;
- a complaint registers itself with both its citizen and its department;
- the lifecycle allows valid moves, refuses skipping steps or going backwards, and records history;
- ratings are allowed only for resolved complaints and must be 1–5;
- overdue detection works against the priority targets;
- department statistics and the best department are calculated correctly;
- deleting a citizen removes their complaints from every dictionary;
- from_text() works, and invalid priorities and future dates are refused;
- the tuple lookups (targets, categories, 16 districts) are correct.

All 10 tests pass (Figure 12). Every menu option was also tested by hand with correct and incorrect input (Figures 5 to 11).

Note: "days open" and "days late" are counted up to the current date, so these figures increase each day the program is run.

## 11. GitHub Submission

The complete solution is published as an open-source repository at: **[PASTE YOUR GITHUB REPOSITORY LINK]**. It contains:

| File | Contents |
|---|---|
| complaint_system.py | The Python source code: all classes, the menu, the demonstration and the unit tests. |

| File | Contents |
|---|---|
| README.md | An explanation of how the solution aligns with DPG standards, how to run it, and example usage with sample input and output for every feature. |
| LICENSE | The MIT open-source licence. |

As required by the brief, the lecturer, **Amandus Benjamin Coker** (amandus.bcoker@limkokwing.edu.sl), has been added as a **collaborator** on the repository through Settings → Collaborators → Add people.

## 12. Assignment Requirements Checklist

| Requirement from the brief | Met? | Where |
|---|---|---|
| Python program using OOP for a real-world problem in one domain | Yes | Sections 1–2 (Government: Citizen Complaint System) |
| Open-source (uploaded to GitHub) | Yes | Sections 8 and 11 |
| Inclusive and accessible design | Yes | Section 8; Figures 5–7 |
| Privacy-respecting (no sensitive personal data) | Yes | Sections 4.1 and 8; anonymous option |
| Modular and reusable | Yes | Sections 3 and 8 |
| Task 1: at least two classes | Yes | Three classes – Section 3.1 |
| Task 1: attributes in each class | Yes | Sections 3.1 and 4 |
| Task 1: methods in each class | Yes | Sections 3.1 and 4 |
| Task 1: object creation and interaction | Yes | Sections 4.2 and 4.3; Figure 3 |
| Task 2: only one data structure (dictionary) | Yes | Section 5 |
| Task 2: function to add records | Yes | add_record() – Section 5.2 |
| Task 2: function to display records | Yes | display_complaints() – Section 5.2; Figure 5 |
| Optional: tuples used correctly (+10 marks) | Yes | Section 5.3 |
| Task 3: at least one instance method | Yes | Section 6.1 |
| Task 3: at least one class method (@classmethod) | Yes | Section 6.2 |
| Static methods (mentioned in the overview) | Yes | Section 6.3 |
| README explaining DPG alignment | Yes | README.md – Section 11 |
| README with example usage (sample input/output) | Yes | README.md – Section 11 |
| Python source code (.py) on GitHub | Yes | complaint_system.py |

| Requirement from the brief | Met? | Where |
|---|---|---|
| Lecturer added as a collaborator | Yes | Section 11 |
| Hardcopy with screenshots of program execution | Yes | Section 9 (Figures 2–12) |
| Format: Tahoma, size 10, 1.15 spacing, justified | Yes | This document |
| Academic integrity and plagiarism policy | Yes | Own work; sources listed in Section 14 |

## 13. Conclusion and Future Improvements

The Citizen Complaint System meets every requirement of the assignment. Three classes (Citizen, Department and Complaint) work closely together: each complaint registers itself with its citizen and department, and departments measure their own performance from the complaints they hold (Task 1). Dictionaries store all records, with functions to add, display, search and delete them, and tuples protect the fixed rules and the complaint history (Task 2). Instance, class and static methods each have a clear role (Task 3). The system follows DPG standards: it is open-source, easy to use, allows anonymous reporting and is modular. Its extra features (deleting complaints and citizens, ranking departments, and flagging overdue complaints) support transparency and accountability in public services.

Future improvements include:

- saving data to a file or database so records last between sessions;
- adding staff log-in so only department officers can change a complaint's status;
- accepting complaints by SMS or USSD for citizens without internet access, and offering the menu in Krio;
- producing monthly reports for each district.

## 14. References


- Python Software Foundation (2026). The Python Tutorial and Python Standard Library documentation (datetime, unittest). https://docs.python.org/3/
- Digital Public Goods Alliance. The Digital Public Goods Standard. https://digitalpublicgoods.net/standard/
- Limkokwing University of Creative Technology (2026). PROG211 Object-Oriented Programming 1, Lectures 1 to 3 and Assignment Brief.

## Appendix A – Full Source Code (complaint_system.py)

Code listings use a monospaced font for readability and to preserve indentation, which is essential in Python.

```
"""
################################################################
#  CITIZEN COMPLAINT SYSTEM                                    #
#  PROG211 Object-Oriented Programming 1 - Individual Work     #
#  Real-World Solutions under DPG Standards                    #
#  Domain: Government                                          #
################################################################

Run
    python complaint_system.py           interactive menu
    python complaint_system.py --demo    automatic demonstration
    python complaint_system.py --test    built-in unit tests

What it does
    Citizens report problems with public services (roads, water,
    electricity, waste, health facilities). Each complaint is sent to
    the responsible department, gets a reference number and a target
    resolution time, and moves through a fixed lifecycle:
        Submitted -> In progress -> Resolved   (or Rejected)
    Citizens can rate resolved complaints, and the system ranks
    departments by how well they respond.

Sections of this file
    1. Fixed data (tuples)
    2. Classes: Citizen, Department, Complaint
    3. Record functions (dictionaries)
    4. Sample data
    5. Input helpers
    6. Menu actions
    7. Menu and demo
    8. Unit tests

Privacy (DPG)
    A citizen may stay completely ANONYMOUS. If they give a name, only
    a first name is kept, together with their district. No surname,
    phone number, email, home address or national ID number is ever
    collected or stored.
"""

import contextlib
import io
import sys
import unittest
from datetime import date


# ============================================================
# 1. FIXED DATA (TUPLES) - values that must never change
# ============================================================

# The 16 districts of Sierra Leone
DISTRICTS = (
    "Bo", "Bombali", "Bonthe", "Falaba", "Kailahun", "Kambia", "Karene", "Kenema",
    "Koinadugu", "Kono", "Moyamba", "Port Loko", "Pujehun", "Tonkolili",
    "Western Area Rural", "Western Area Urban",
)

# (category code, description, responsible department code, department name)
CATEGORIES = (
    ("ROAD",   "Roads and potholes",     "DPW", "Public Works Department"),
    ("WATER",  "Water supply",           "DWS", "Water Services Department"),
    ("POWER",  "Electricity supply",     "DEL", "Electricity Department"),
    ("WASTE",  "Waste collection",       "DSN", "Sanitation Department"),
    ("HEALTH", "Health facility service", "DHS", "Health Services Department"),
)

# (priority, target number of days to resolve)
PRIORITIES = (
    ("High",   3),
    ("Medium", 7),
    ("Low",   14),
)
```

```
# Allowed status changes: (from status, to status)
STATUS_FLOW = (
    ("Submitted",   "In progress"),
    ("Submitted",   "Rejected"),
    ("In progress", "Resolved"),
    ("In progress", "Rejected"),
)
OPEN_STATUSES = ("Submitted", "In progress")

RATING_RANGE = (1, 5)   # citizen satisfaction stars


def target_days(priority):
    for name, days in PRIORITIES:            # tuple unpacking
        if name == priority:
            return days
    raise ValueError("Priority must be High, Medium or Low.")


def category_details(code):
    """Return (code, description, department code, department name) for a category."""
    for entry in CATEGORIES:
        if entry[0] == code.strip().upper():
            return entry
    raise ValueError("Category must be one of: " + ", ".join(c[0] for c in CATEGORIES) + ".")


# ============================================================
# 2. CLASSES
# ============================================================

class Citizen:
    """A person who files complaints. May be anonymous."""

    total_citizens = 0
    next_number = 1

    def __init__(self, citizen_id, first_name, district):
        if not Citizen.is_valid_district(district):
            raise ValueError("Please choose one of the 16 districts of Sierra Leone.")
        self.citizen_id = citizen_id.upper()
        self.first_name = first_name.strip().capitalize() or "Anonymous"
        self.district = Citizen.normalise_district(district)
        self.complaints = {}                 # {reference: Complaint}
        Citizen.total_citizens += 1

    # ---------- instance methods ----------
    def add_complaint(self, complaint):
        self.complaints[complaint.reference] = complaint

    def open_complaints(self):
        count = 0
        for complaint in self.complaints.values():
            if complaint.is_open():
                count += 1
        return count

    def display_info(self):
        print(f"  Citizen ID : {self.citizen_id}")
        print(f"  Name       : {self.first_name}")
        print(f"  District   : {self.district}")
        print(f"  Complaints : {len(self.complaints)} filed, {self.open_complaints()} still
open")

    def __str__(self):
        return f"{self.citizen_id} ({self.first_name}, {self.district})"

    # ---------- class methods ----------
    @classmethod
    def register(cls, first_name, district):
        """Create a citizen with an automatic ID such as C004."""
        citizen = cls(f"C{cls.next_number:03d}", first_name, district)
        cls.next_number += 1
        return citizen

    @classmethod
    def register_anonymous(cls, district):
        """Create an anonymous citizen - only the district is kept."""
        return cls.register("Anonymous", district)
```

```
    @classmethod
    def get_total_citizens(cls):
        return cls.total_citizens

    @classmethod
    def record_removal(cls):
        cls.total_citizens -= 1

    # ---------- static methods ----------
    @staticmethod
    def normalise_district(text):
        for district in DISTRICTS:
            if district.lower() == text.strip().lower():
                return district
        return None

    @staticmethod
    def is_valid_district(text):
        return Citizen.normalise_district(text) is not None


class Department:
    """A government department that receives and resolves complaints."""

    def __init__(self, code, name):
        self.code = code
        self.name = name
        self.complaints = {}                # {reference: Complaint}

    # ---------- instance methods ----------
    def receive(self, complaint):
        self.complaints[complaint.reference] = complaint

    def count_by_status(self, status):
        count = 0
        for complaint in self.complaints.values():
            if complaint.status == status:
                count += 1
        return count

    def resolution_rate(self):
        """Percentage of this department's complaints that are resolved."""
        if not self.complaints:
            return 0
        return round(self.count_by_status("Resolved") / len(self.complaints) * 100)

    def average_rating(self):
        total, rated = 0, 0
        for complaint in self.complaints.values():
            if complaint.rating is not None:
                total += complaint.rating
                rated += 1
        return round(total / rated, 1) if rated else 0.0

    def average_days_to_resolve(self):
        total, resolved = 0, 0
        for complaint in self.complaints.values():
            if complaint.status == "Resolved":
                total += complaint.days_open()
                resolved += 1
        return round(total / resolved, 1) if resolved else 0.0

    def on_time_rate(self):
        """Percentage of resolved complaints that were resolved within their target days."""
        on_time, resolved = 0, 0
        for complaint in self.complaints.values():
            if complaint.status == "Resolved":
                resolved += 1
                if complaint.days_open() <= target_days(complaint.priority):
                    on_time += 1
        return round(on_time / resolved * 100) if resolved else 0

    def performance_score(self):
        """Out of 100: 50% resolution rate + 30% citizen rating + 20% resolved on time."""
        rating_percent = self.average_rating() / RATING_RANGE[1] * 100
        return round(0.5 * self.resolution_rate() + 0.3 * rating_percent
                     + 0.2 * self.on_time_rate())
```

```
    def statistics(self):
        """Return a TUPLE:
        (total, open, resolved, resolution %, avg rating, avg days, on-time %, score)."""
        open_count = self.count_by_status("Submitted") + self.count_by_status("In progress")
        return (len(self.complaints), open_count, self.count_by_status("Resolved"),
                self.resolution_rate(), self.average_rating(),
                self.average_days_to_resolve(), self.on_time_rate(), self.performance_score())

    # ---------- class method ----------
    @classmethod
    def from_category(cls, category_entry):
        """Create the department named in one CATEGORIES tuple."""
        _, _, code, name = category_entry
        return cls(code, name)

    # ---------- static method ----------
    @staticmethod
    def find_best(departments):
        """Return the department with the highest performance score (or None)."""
        best = None
        for department in departments.values():
            if not department.complaints:
                continue
            if best is None or department.performance_score() > best.performance_score():
                best = department
        return best


class Complaint:
    """One complaint about a public service."""

    total_complaints = 0
    next_number = 1

    def __init__(self, citizen, department, category, description, priority, date_filed):
        if not description.strip():
            raise ValueError("Please describe the problem.")
        self.priority = Complaint.normalise_priority(priority)
        if isinstance(date_filed, str):
            date_filed = Complaint.parse_date(date_filed)
        self.reference = f"CMP-{Complaint.next_number:04d}"
        Complaint.next_number += 1
        Complaint.total_complaints += 1
        self.citizen = citizen               # Citizen OBJECT
        self.department = department         # Department OBJECT
        self.category = category.upper()
        self.description = description.strip()
        self.date_filed = date_filed
        self.status = "Submitted"
        self.date_closed = None
        self.rating = None
        # history is a tuple of (date, status, note) tuples
        self.history = ((date_filed, "Submitted", "Complaint received"),)
        # objects interacting: register with the citizen AND the department
        citizen.add_complaint(self)
        department.receive(self)

    # ---------- instance methods ----------
    def is_open(self):
        return self.status in OPEN_STATUSES

    def days_open(self, today=None):
        end = self.date_closed or today or date.today()
        return (end - self.date_filed).days

    def is_overdue(self, today=None):
        return self.is_open() and self.days_open(today) > target_days(self.priority)

    def update_status(self, new_status, note, when=None):
        """Move the complaint to a new status, following STATUS_FLOW."""
        new_status = new_status.strip().capitalize()
        if (self.status, new_status) not in STATUS_FLOW:
            raise ValueError(f"A complaint that is '{self.status}' cannot become
'{new_status}'.")
        when = when or date.today()
        if isinstance(when, str):
            when = Complaint.parse_date(when)
        if when < self.date_filed:
            raise ValueError("The update date cannot be before the complaint was filed.")
```

```
        old = self.status
        self.status = new_status
        if not self.is_open():
            self.date_closed = when
        self.history = self.history + ((when, new_status, note.strip() or "-"),)
        print(f"  {self.reference}: {old} -> {new_status}")

    def rate(self, stars):
        """Citizen satisfaction rating, only allowed once the complaint is resolved."""
        if self.status != "Resolved":
            raise ValueError("Only resolved complaints can be rated.")
        if not Complaint.is_valid_rating(stars):
            low, high = RATING_RANGE
            raise ValueError(f"Rating must be a whole number from {low} to {high}.")
        self.rating = stars

    def display_row(self):
        flag = "  OVERDUE" if self.is_overdue() else ""
        print(f"  {self.reference:<10}{self.category:<8}{self.citizen.district:<20}"
              f"{self.priority:<9}{self.status:<13}{self.days_open():>4} d{flag}")

    def display_details(self):
        line = "#" * 64
        stars = ("*" * self.rating) if self.rating else "not rated"
        print(line)
        print(f"  COMPLAINT {self.reference}")
        print(line)
        print(f"  Filed by    : {self.citizen}")
        print(f"  Category    : {category_details(self.category)[1]}")
        print(f"  Department  : {self.department.name}")
        print(f"  Problem     : {self.description}")
        print(f"  Priority    : {self.priority} (target {target_days(self.priority)} days)")
        print(f"  Status      : {self.status}{'  ** OVERDUE **' if self.is_overdue() else ''}")
        print(f"  Days open   : {self.days_open()}")
        print(f"  Rating      : {stars}")
        print("  History:")
        for when, status, note in self.history:          # tuple unpacking
            print(f"    {when.isoformat()}  {status:<12} {note}")
        print(line)

    # ---------- class methods ----------
    @classmethod
    def from_text(cls, text, citizens, departments):
        """Create a complaint from 'C001, ROAD, Large pothole, High, 2026-09-01'."""
        citizen_id, category, description, priority, when = (p.strip() for p in text.split(","))
        citizen = find_record(citizens, citizen_id, "citizen")
        _, _, dept_code, _ = category_details(category)
        return cls(citizen, departments[dept_code], category, description, priority, when)

    @classmethod
    def get_total_complaints(cls):
        return cls.total_complaints

    @classmethod
    def record_removal(cls, count=1):
        cls.total_complaints -= count

    # ---------- static methods ----------
    @staticmethod
    def normalise_priority(text):
        for name, _ in PRIORITIES:
            if name.lower() == text.strip().lower():
                return name
        raise ValueError("Priority must be High, Medium or Low.")

    @staticmethod
    def is_valid_rating(stars):
        low, high = RATING_RANGE
        return isinstance(stars, int) and low <= stars <= high

    @staticmethod
    def parse_date(text):
        try:
            when = date.fromisoformat(text.strip())
        except ValueError:
            raise ValueError("Dates must be written as YYYY-MM-DD, e.g. 2026-09-30.")
        if when > date.today():
            raise ValueError("The date cannot be in the future.")
        return when
```

```
# ============================================================
# 3. RECORD FUNCTIONS (DICTIONARIES)
# The DICTIONARY is the one data structure used to store objects:
#     citizens    = {"C001": <Citizen>, ...}
#     departments = {"DPW": <Department>, ...}
#     complaints  = {"CMP-0001": <Complaint>, ...}
# ============================================================

def add_record(records, key, item):
    """Add an object to a dictionary (duplicate keys are refused)."""
    if key in records:
        raise ValueError(f"{key} already exists.")
    records[key] = item
    return item


def find_record(records, key, label="record"):
    key = key.strip().upper()
    if key not in records:
        raise ValueError(f"No {label} found with ID {key}.")
    return records[key]


def withdraw_complaint(complaints, reference):
    """Delete one complaint everywhere it is stored."""
    complaint = find_record(complaints, reference, "complaint")
    del complaints[complaint.reference]
    del complaint.citizen.complaints[complaint.reference]
    del complaint.department.complaints[complaint.reference]
    Complaint.record_removal()
    return complaint


def delete_citizen(citizens, complaints, citizen_id):
    """Delete a citizen and every complaint they filed."""
    citizen = find_record(citizens, citizen_id, "citizen")
    for reference in tuple(citizen.complaints):        # copy keys before deleting
        withdraw_complaint(complaints, reference)
    del citizens[citizen.citizen_id]
    Citizen.record_removal()
    return citizen


def display_complaints(complaints):
    print(f"\n  ALL COMPLAINTS ({len(complaints)})")
    print("  " + "-" * 72)
    print(f"  {'Reference':<10}{'Type':<8}{'District':<20}{'Priority':<9}{'Status':<13}
{'Open':>6}")
    print("  " + "-" * 72)
    for complaint in complaints.values():
        complaint.display_row()


def search_complaints(complaints, text):
    """Show complaints whose reference, category, district, status or description matches."""
    text = text.lower()
    found = 0
    for c in complaints.values():
        if (text in c.reference.lower() or text in c.category.lower()
                or text in c.citizen.district.lower() or text in c.status.lower()
                or text in c.description.lower()):
            c.display_row()
            found += 1
    return found


# ============================================================
# 4. SAMPLE DATA (fictional)
# ============================================================

def load_sample_data():
    departments = {}
    for entry in CATEGORIES:
        department = Department.from_category(entry)          # class method
        add_record(departments, department.code, department)

    citizens = {}
```

```
    for name, district in (("Aminata", "Western Area Urban"), ("Mohamed", "Bo"),
                           ("", "Kenema"), ("Fatmata", "Bombali")):
        citizen = Citizen.register(name, district) if name else
Citizen.register_anonymous(district)
        add_record(citizens, citizen.citizen_id, citizen)

    complaints = {}
    for text in (
        "C001, ROAD, Large pothole on main road near the market, High, 2026-09-01",
        "C001, WASTE, Rubbish not collected for two weeks, Medium, 2026-09-10",
        "C002, WATER, No running water in the street, High, 2026-09-15",
        "C002, POWER, Street lights not working, Low, 2026-08-20",
        "C003, HEALTH, Clinic closed during opening hours, Medium, 2026-09-25",
        "C003, ROAD, Bridge railing broken, High, 2026-10-01",
        "C004, WASTE, Blocked drainage after rain, Medium, 2026-09-05",
        "C004, WATER, Public tap leaking, Low, 2026-10-06",
    ):
        complaint = Complaint.from_text(text, citizens, departments)    # class method
        add_record(complaints, complaint.reference, complaint)

    updates = (
        ("CMP-0001", "In progress", "Inspection team sent", "2026-09-02"),
        ("CMP-0001", "Resolved", "Pothole filled", "2026-09-04"),
        ("CMP-0002", "In progress", "Truck scheduled", "2026-09-12"),
        ("CMP-0002", "Resolved", "Area cleared", "2026-09-14"),
        ("CMP-0003", "In progress", "Pipe repair started", "2026-09-16"),
        ("CMP-0004", "In progress", "Lights ordered", "2026-08-25"),
        ("CMP-0004", "Resolved", "New lights fitted", "2026-09-05"),
        ("CMP-0007", "Rejected", "Private drain - owner responsible", "2026-09-07"),
    )
    for reference, status, note, when in updates:
        _quiet(complaints[reference].update_status, status, note, when)
    for reference, stars in (("CMP-0001", 5), ("CMP-0002", 4), ("CMP-0004", 3)):
        complaints[reference].rate(stars)
    return citizens, departments, complaints


def _quiet(function, *args):
    """Run a function without printing its messages (used for sample data)."""
    with contextlib.redirect_stdout(io.StringIO()):
        function(*args)


# ============================================================
# 5. INPUT HELPERS
# ============================================================

def ask(prompt):
    return input(prompt).strip()


def ask_until_valid(prompt, convert, problem):
    while True:
        try:
            return convert(ask(prompt))
        except ValueError:
            print(f"  Problem: {problem}")


def valid_district(text):
    district = Citizen.normalise_district(text)
    if district is None:
        raise ValueError
    return district


def non_empty(text):
    if not text.strip():
        raise ValueError
    return text.strip()


def valid_rating(text):
    stars = int(text)
    if not Complaint.is_valid_rating(stars):
        raise ValueError
    return stars
```

```
# ============================================================
# 6. MENU ACTIONS
# ============================================================

def file_complaint(citizens, departments, complaints):
    citizen = find_record(citizens, ask("  Your citizen ID (e.g. C002): "), "citizen")
    print("  Categories:")
    for code, text, _, dept_name in CATEGORIES:
        print(f"    {code:<7} {text:<25} -> {dept_name}")
    category = ask_until_valid("  Category code: ", lambda t: category_details(t)[0],
                               "please type one of the category codes shown.")
    description = ask_until_valid("  Describe the problem: ", non_empty,
                                  "the description cannot be empty.")
    priority = ask_until_valid("  Priority (High/Medium/Low): ", Complaint.normalise_priority,
                               "please type High, Medium or Low.")
    _, _, dept_code, dept_name = category_details(category)
    complaint = Complaint(citizen, departments[dept_code], category, description,
                          priority, date.today())
    add_record(complaints, complaint.reference, complaint)
    print(f"  [OK] Complaint {complaint.reference} sent to the {dept_name}.")
    print(f"       Target: resolved within {target_days(priority)} days. Keep your reference
number.")


def update_complaint(complaints):
    complaint = find_record(complaints, ask("  Complaint reference (e.g. CMP-0003): "),
"complaint")
    print(f"  Current status: {complaint.status}")
    allowed = ", ".join(to for frm, to in STATUS_FLOW if frm == complaint.status)
    if not allowed:
        raise ValueError(f"This complaint is already {complaint.status} and cannot change.")
    new_status = ask(f"  New status ({allowed}): ")
    note = ask("  Note for the citizen: ")
    complaint.update_status(new_status, note)


def rate_complaint(complaints):
    complaint = find_record(complaints, ask("  Complaint reference: "), "complaint")
    if complaint.status != "Resolved":
        raise ValueError("Only resolved complaints can be rated.")
    stars = ask_until_valid("  How satisfied are you? (1 = very poor, 5 = excellent): ",
                            valid_rating, "please type a whole number from 1 to 5.")
    complaint.rate(stars)
    print(f"  [OK] Thank you. {complaint.reference} rated {'*' * stars} ({stars}/5).")


def track_complaint(complaints):
    complaint = find_record(complaints, ask("  Complaint reference: "), "complaint")
    print()
    complaint.display_details()


def show_districts():
    """Print the 16 districts in rows of four so they fit on screen."""
    print("  Districts:")
    for start in range(0, len(DISTRICTS), 4):
        print("    " + "".join(f"{name:<20}" for name in DISTRICTS[start:start + 4]))


def register_citizen(citizens):
    anonymous = ask("  Stay anonymous? (Y/N): ").upper() == "Y"
    name = "" if anonymous else ask("  First name only: ")
    show_districts()
    district = ask_until_valid("  District: ", valid_district,
                               "please type one of the districts listed.")
    citizen = Citizen.register_anonymous(district) if anonymous else Citizen.register(name,
district)
    add_record(citizens, citizen.citizen_id, citizen)
    print(f"  [OK] Registered {citizen}. Use ID {citizen.citizen_id} to file complaints.")


def withdraw(complaints):
    complaint = find_record(complaints, ask("  Reference of the complaint to withdraw: "),
"complaint")
    print(f"  {complaint.reference}: {complaint.description} ({complaint.status})")
    if ask("  Type YES to withdraw and delete it: ").upper() != "YES":
        print("  Cancelled - nothing was deleted.")
        return
    withdraw_complaint(complaints, complaint.reference)
```

```
    print(f"  [OK] {complaint.reference} withdrawn. {len(complaints)} complaint(s) remain.")


def remove_citizen(citizens, complaints):
    citizen = find_record(citizens, ask("  Citizen ID to delete: "), "citizen")
    citizen.display_info()
    if ask("  Type YES to delete this citizen and ALL their complaints: ").upper() != "YES":
        print("  Cancelled - nothing was deleted.")
        return
    count = len(citizen.complaints)
    delete_citizen(citizens, complaints, citizen.citizen_id)
    print(f"  [OK] Deleted {citizen.citizen_id} and {count} complaint(s).")


def search(complaints):
    text = ask("  Search (reference, category, district, status or word): ")
    if not text:
        raise ValueError("Please type something to search for.")
    print()
    if search_complaints(complaints, text) == 0:
        print("  No complaints matched.")


def show_best_department(departments):
    best = Department.find_best(departments)                    # static method
    print("\n  BEST-PERFORMING DEPARTMENT")
    print("  " + "-" * 50)
    if best is None:
        print("  No complaints have been filed yet.")
        return
    total, open_count, resolved, rate, rating, days, on_time, score = best.statistics()
    print(f"  {best.name} ({best.code})")
    print(f"  Performance score : {score}/100")
    print(f"  Resolved          : {resolved} of {total} ({rate}%)")
    print(f"  Citizen rating    : {rating}/5")
    print(f"  Average time      : {days} days to resolve, {on_time}% on time")


def show_department_report(departments):
    print("\n  DEPARTMENT PERFORMANCE REPORT")
    print("  " + "-" * 80)
    print(f"  {'Department':<28}{'Total':>6}{'Open':>5}{'Done':>5}{'Rate':>6}"
          f"{'Stars':>6}{'Days':>6}{'OnTime':>7}{'Score':>7}")
    print("  " + "-" * 80)
    for department in departments.values():
        total, open_count, resolved, rate, rating, days, on_time, score = department.statistics()
        print(f"  {department.name:<28}{total:>6}{open_count:>5}{resolved:>5}{rate:>5}%"
              f"{rating:>6}{days:>6}{on_time:>6}%{score:>7}")
    print("  Score = 50% resolution rate + 30% citizen rating + 20% resolved on time")


def show_overdue(complaints):
    print(f"\n  OVERDUE COMPLAINTS (open longer than their target, as of {date.today()})")
    print("  " + "-" * 72)
    found = 0
    for complaint in complaints.values():
        if complaint.is_overdue():
            found += 1
            late = complaint.days_open() - target_days(complaint.priority)
            print(f"  {complaint.reference}  {complaint.department.name:<28}"
                  f"{complaint.priority:<7} {late} day(s) late")
    print(f"  {found} complaint(s) need urgent action." if found else "  Nothing is overdue.")


def show_summary(citizens, complaints):
    counts = ()
    for status in ("Submitted", "In progress", "Resolved", "Rejected"):
        count = 0
        for complaint in complaints.values():
            if complaint.status == status:
                count += 1
        counts = counts + ((status, count),)
    print("\n  SYSTEM SUMMARY")
    print("  " + "-" * 40)
    print(f"  Citizens registered : {Citizen.get_total_citizens()}")
    print(f"  Complaints on file  : {Complaint.get_total_complaints()}")
    for status, count in counts:
        print(f"    {status:<12}: {count}")
    print("  Priority targets (fixed tuple): " +
```

```
          ", ".join(f"{name} {days} days" for name, days in PRIORITIES))


# ============================================================
# 7. MENU AND DEMO
# ============================================================

MENU = (
    "View all complaints",
    "File a new complaint",
    "Update a complaint's status",
    "Rate a resolved complaint",
    "Track a complaint",
    "Register a citizen",
    "Withdraw (delete) a complaint",
    "Delete a citizen",
    "Search complaints",
    "Best-performing department",
    "Department performance report",
    "Overdue complaints",
    "System summary",
)


def main():
    citizens, departments, complaints = load_sample_data()
    print("#" * 60)
    print("#{:^58}#".format("CITIZEN COMPLAINT SYSTEM"))
    print("#{:^58}#".format("Your voice on public services - anonymous if you wish"))
    print("#" * 60)
    while True:
        print()
        for number, label in enumerate(MENU, start=1):
            print(f"  {number:>2}) {label}")
        print("   0) Exit")
        choice = ask("  Option: ")
        try:
            if choice == "1":
                display_complaints(complaints)
            elif choice == "2":
                file_complaint(citizens, departments, complaints)
            elif choice == "3":
                update_complaint(complaints)
            elif choice == "4":
                rate_complaint(complaints)
            elif choice == "5":
                track_complaint(complaints)
            elif choice == "6":
                register_citizen(citizens)
            elif choice == "7":
                withdraw(complaints)
            elif choice == "8":
                remove_citizen(citizens, complaints)
            elif choice == "9":
                search(complaints)
            elif choice == "10":
                show_best_department(departments)
            elif choice == "11":
                show_department_report(departments)
            elif choice == "12":
                show_overdue(complaints)
            elif choice == "13":
                show_summary(citizens, complaints)
            elif choice == "0":
                print("  Thank you for helping improve public services. Goodbye!")
                break
            else:
                print("  Problem: please type one of the numbers shown.")
        except ValueError as problem:
            print(f"  Problem: {problem}")


def step(text):
    print(f"\n### {text}")


def run_demo():
    step("1. Creating objects (Task 1)")
    citizens, departments, complaints = load_sample_data()
```

```
    for citizen in citizens.values():
        print(f"  {citizen}")

    step("2. Complaints stored in a dictionary (Task 2)")
    display_complaints(complaints)

    step("3. Class methods: Citizen.register_anonymous() and Complaint.from_text()")
    newcomer = Citizen.register_anonymous("kono")
    add_record(citizens, newcomer.citizen_id, newcomer)
    complaint = Complaint.from_text(f"{newcomer.citizen_id}, POWER, Transformer sparking at
night, High, 2026-10-10",
                                    citizens, departments)
    add_record(complaints, complaint.reference, complaint)
    print(f"  {newcomer} filed {complaint.reference} -> {complaint.department.name}")

    step("4. Instance methods: update_status() follows the lifecycle, rate() records
satisfaction")
    complaint.update_status("In progress", "Engineer assigned", "2026-10-11")
    complaint.update_status("Resolved", "Transformer replaced", "2026-10-12")
    complaint.rate(5)
    for bad in (lambda: complaint.update_status("Submitted", "x"),
                lambda: complaints["CMP-0003"].rate(4),
                lambda: Citizen.register("Musa", "Atlantis")):
        try:
            bad()
        except ValueError as problem:
            print(f"  Problem: {problem}")

    step("5. Static methods")
    print(f"  Complaint.is_valid_rating(6)        -> {Complaint.is_valid_rating(6)}")
    print(f"  Citizen.is_valid_district('BO')     -> {Citizen.is_valid_district('BO')}")
    print(f"  Complaint.normalise_priority('low') -> {Complaint.normalise_priority('low')}")

    step("6. Object interaction: one complaint seen by citizen and department")
    complaint.display_details()
    print(f"  {newcomer.citizen_id} has {len(newcomer.complaints)} complaint(s); "
          f"{complaint.department.code} has {len(complaint.department.complaints)}.")

    step("7. Best department, performance report and overdue list")
    show_best_department(departments)
    show_department_report(departments)
    show_overdue(complaints)

    step("8. Deleting: withdraw a complaint, then delete a citizen")
    withdraw_complaint(complaints, "CMP-0008")
    removed = delete_citizen(citizens, complaints, "C002")
    print(f"  [OK] CMP-0008 withdrawn; {removed.citizen_id} and their complaints deleted.")
    show_summary(citizens, complaints)


# ============================================================
# 8. UNIT TESTS  (python complaint_system.py --test)
# ============================================================

class ComplaintSystemTests(unittest.TestCase):

    def setUp(self):
        self.road = Department("DPW", "Public Works Department")
        self.citizen = Citizen.register("adama", "bo")
        self.complaint = Complaint(self.citizen, self.road, "ROAD", "Pothole", "high", "2026-09-
01")

    def test_citizen_privacy_and_district(self):
        self.assertEqual((self.citizen.first_name, self.citizen.district), ("Adama", "Bo"))
        anonymous = Citizen.register_anonymous("western area urban")
        self.assertEqual(anonymous.first_name, "Anonymous")
        with self.assertRaises(ValueError):
            Citizen.register("Musa", "Atlantis")

    def test_complaint_registers_with_both_objects(self):
        ref = self.complaint.reference
        self.assertIn(ref, self.citizen.complaints)
        self.assertIn(ref, self.road.complaints)

    def test_status_lifecycle(self):
        self.complaint.update_status("In progress", "Team sent", "2026-09-02")
        self.complaint.update_status("Resolved", "Fixed", "2026-09-03")
        self.assertEqual(self.complaint.days_open(), 2)
        with self.assertRaises(ValueError):
```

```
            self.complaint.update_status("In progress", "again")
        self.assertEqual(len(self.complaint.history), 3)

    def test_cannot_skip_or_go_back(self):
        with self.assertRaises(ValueError):
            self.complaint.update_status("Resolved", "skip")

    def test_rating_rules(self):
        with self.assertRaises(ValueError):
            self.complaint.rate(5)                    # not resolved yet
        self.complaint.update_status("In progress", "", "2026-09-02")
        self.complaint.update_status("Resolved", "", "2026-09-03")
        with self.assertRaises(ValueError):
            self.complaint.rate(9)
        self.complaint.rate(4)
        self.assertEqual(self.road.average_rating(), 4.0)

    def test_overdue(self):
        self.assertTrue(self.complaint.is_overdue(today=date(2026, 9, 10)))
        self.assertFalse(self.complaint.is_overdue(today=date(2026, 9, 3)))

    def test_department_statistics_and_best(self):
        self.complaint.update_status("In progress", "", "2026-09-02")
        self.complaint.update_status("Resolved", "", "2026-09-03")
        self.complaint.rate(5)
        water = Department("DWS", "Water Services Department")
        Complaint(self.citizen, water, "WATER", "No water", "Low", "2026-09-01")
        self.assertEqual(self.road.statistics(), (1, 0, 1, 100, 5.0, 2.0, 100, 100))
        self.assertIs(Department.find_best({"DPW": self.road, "DWS": water}), self.road)

    def test_withdraw_and_delete_citizen(self):
        complaints = {self.complaint.reference: self.complaint}
        citizens = {self.citizen.citizen_id: self.citizen}
        delete_citizen(citizens, complaints, self.citizen.citizen_id)
        self.assertEqual((len(citizens), len(complaints), len(self.road.complaints)), (0, 0, 0))

    def test_from_text_and_validation(self):
        citizens = {self.citizen.citizen_id: self.citizen}
        departments = {"DPW": self.road}
        made = Complaint.from_text(f"{self.citizen.citizen_id}, road, Crack, Medium, 2026-09-05",
                                   citizens, departments)
        self.assertEqual((made.category, made.priority), ("ROAD", "Medium"))
        with self.assertRaises(ValueError):
            Complaint(self.citizen, self.road, "ROAD", "x", "Urgent", "2026-09-05")
        with self.assertRaises(ValueError):
            Complaint.parse_date("2099-01-01")

    def test_tuple_lookups(self):
        self.assertEqual(target_days("Medium"), 7)
        self.assertEqual(category_details("water")[2], "DWS")
        self.assertEqual(len(DISTRICTS), 16)


if __name__ == "__main__":
    if "--test" in sys.argv:
        unittest.main(argv=[sys.argv[0], "-v"])
    elif "--demo" in sys.argv:
        run_demo()
    else:
        main()
```